

RV UNIVERSITY

School of Computer Science and Engineering

Laboratory Record Book

OPERATING SYSTEMS - [CS1103]

Bachelor of Technology (B.Tech.) (Hons.)

Academic Year 2025-26

Name: Francis Reuben R

Roll Number: 1RUA25CSE0168

Section: B-50 | Second Semester (2025-29)

Vision & Mission

Vision

The School of Computer Science and Engineering at RV University envisions the Operating Systems course as a cornerstone of computing education, equipping students with a deep understanding of system architecture, resource management, and process coordination. This course fosters a strong foundation in theoretical principles and practical applications...

Mission

- Fundamental Understanding - An OS itself is a rich set of concepts involving process management, memory management, file systems, and security.
- Hands-On Learning - Students will have hands-on experiences with lab experiments, coding assignments, and project work involving actual systems.
- Industry Relevance - Teaching concerning current industry practices offers students some understanding of topics in virtualization, cloud computing, and cybersecurity.
- Research & Innovation - A research perspective fosters a contribution towards solving system-level problems.
- Performance Optimization - Students learn how to analyze and optimize the OS component for efficiency, scalability, and reliability.
- Security & Reliability - Systems must be designed to be secure and to tolerate failures.

Course Outcomes (COs)

- CO1: Understand the structural components and core functionalities of an operating system.
- CO2: Apply the process management techniques and scheduling algorithms to ensure efficient execution and resource allocation.
- CO3: Apply synchronization techniques and design solutions to prevent or resolve deadlocks in concurrent systems.
- CO4: Illustrate the memory management strategies, principles of file system design, and security mechanisms.

Laboratory Programs & Solutions

Note: As per requirements, standard Linux/POSIX System Calls (e.g., fork, exec, pthread) are presented. Since Turbo C++ is a 16-bit DOS compiler that inherently lacks POSIX OS capabilities, a "Turbo C++ styled" terminal output has been simulated for aesthetic consistency, while utilizing accurate GCC-style execution flows to maintain high accuracy and prevent functional hallucination.

1.1) Exploring basic Linux commands and Shell programs.

```
#!/bin/bash
# Basic Shell Program
echo "Current Directory:"
pwd
echo "List of Files:"
ls -l
echo "Hello, Operating Systems Lab!"
```

```
C:\TC\BIN> bash script.sh
Current Directory:
/home/user/os_lab
List of Files:
total 4
-rwxr-xr-x 1 user user 114 May 2 08:15 script.sh
Hello, Operating Systems Lab!
```

1.2) Execute the C program using Command Line Arguments (Add, Avg, Min).

```
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {
    if (argc < 2) {
        printf("Please provide numbers as command line arguments.\n");
        return 1;
    }
    int sum = 0, min = atoi(argv[1]);
    for(int i = 1; i < argc; i++) {
        int val = atoi(argv[i]);
        sum += val;
        if(val < min) min = val;
    }
    printf("Sum: %d\n", sum);
    printf("Average: %.2f\n", (float)sum / (argc - 1));
    printf("Minimum: %d\n", min);
    return 0;
}
```

```
C:\TC\BIN> CMDARGS.EXE 10 25 5 40
Sum: 80
Average: 20.00
Minimum: 5
```

1.3) Write a program in C to print the environment variable.

```
#include <stdio.h>

extern char **environ;

int main() {
    int count = 0;
    for (char **env = environ; *env != 0 && count < 5; env++) {
        printf("%s\n", *env);
        count++;
    }
    printf("... (truncated for brevity)\n");
    return 0;
}
```

```
C:\TC\BIN> ENVVAR.EXE
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
USER=user
LANG=en_US.UTF-8
HOME=/home/user
SHELL=/bin/bash
... (truncated for brevity)
```

2. Execute C programs to implement Process system calls using fork, exec, wait.

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();

    if (pid < 0) {
        printf("Fork failed.\n");
    } else if (pid == 0) {
        printf("Child Process: PID=%d. Executing 'ls' command...\n",
getpid());
        execlp("ls", "ls", "-l", NULL);
    } else {
        wait(NULL);
        printf("Parent Process: PID=%d. Child terminated.\n", getpid());
    }
    return 0;
}
```

```
C:\TC\BIN> FORKEXEC.EXE
Child Process: PID=1452. Executing 'ls' command...
total 16
-rwxr-xr-x 1 user user 8432 May 2 08:20 FORKEXEC.EXE
-rw-r--r-- 1 user user 340 May 2 08:20 forkexec.c
Parent Process: PID=1451. Child terminated.
```

3.1) First Come First Served (FCFS) algorithm

```
#include <stdio.h>

int main() {
    int n = 3, bt[] = {10, 5, 8}, wt[3], tat[3];
    float awt = 0, atat = 0;

    wt[0] = 0;
    for(int i = 1; i < n; i++) {
        wt[i] = bt[i-1] + wt[i-1];
    }
    for(int i = 0; i < n; i++) {
        tat[i] = bt[i] + wt[i];
        awt += wt[i];
        atat += tat[i];
    }

    printf("Process\tBurst\tWaiting\tTurnaround\n");
    for(int i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\n", i+1, bt[i], wt[i], tat[i]);
    }
    printf("Avg Waiting Time: %.2f\n", awt/n);
    printf("Avg Turnaround Time: %.2f\n", atat/n);
    return 0;
}
```

```
C:\TC\BIN> FCFS.EXE
Process Burst    Waiting Turnaround
P1      10       0      10
P2       5      10      15
P3       8      15      23
Avg Waiting Time: 8.33
Avg Turnaround Time: 16.00
```

4. Thread Creation and Management using the POSIX pthread library.

```
#include <stdio.h>
#include <pthread.h>

void* threadFunc(void* arg) {
    char* msg = (char*)arg;
    printf("Inside %s\n", msg);
    return NULL;
}

int main() {
    pthread_t t1, t2;
    printf("Creating threads...\n");

    pthread_create(&t1, NULL, threadFunc, "Thread 1");
    pthread_create(&t2, NULL, threadFunc, "Thread 2");

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("Threads execution completed.\n");
    return 0;
}
```

```
C:\TC\BIN> PTHREAD.EXE
Creating threads...
Inside Thread 1
Inside Thread 2
Threads execution completed.
```


5. Execute a C Program for IPC using the pipe() function.

```
#include <stdio.h>
#include <unistd.h>
#include <string.h>

int main() {
    int fd[2];
    char buffer[100];
    pipe(fd);

    if (fork() == 0) {
        close(fd[0]);
        char msg[] = "IPC via Pipe Successful!";
        write(fd[1], msg, strlen(msg) + 1);
        printf("Child wrote to pipe.\n");
    } else {
        close(fd[1]);
        read(fd[0], buffer, sizeof(buffer));
        printf("Parent read from pipe: %s\n", buffer);
    }
    return 0;
}
```

```
C:\TC\BIN> PIPE.EXE
Child wrote to pipe.
Parent read from pipe: IPC via Pipe Successful!
```